

Linux Init Systems - Complete Notes

I. System V (Sys V) Init Overview

Purpose and Characteristics

- **Main purpose:** Start and stop essential processes on the system
- **Three major init implementations:** System V, Upstart, and systemd
- **Sequential processing:** Starts and stops processes one at a time in order
- **Dependency management:** Ensures services start in correct order (foo-a before foo-b)

Identification (Sys V)

- **How to identify:** Check if `/etc/inittab` file exists

Advantages and Disadvantages

- **Pros:** Easy to solve dependencies due to sequential nature
- **Cons:** Poor performance (one process at a time)

Runlevels (0-6)

- **0:** Shutdown
- **1:** Single User Mode
- **2:** Multiuser mode without networking
- **3:** Multiuser mode with networking
- **4:** Unused
- **5:** Multiuser mode with networking and GUI
- **6:** Reboot

Script Locations and Naming

- **Location:** `/etc/rc.d/rc[runlevel number].d/` or `/etc/init.d`
- **Naming convention:**
 - Scripts starting with **S** (start) run on startup
 - Scripts starting with **K** (kill) run on shutdown
 - Numbers indicate execution sequence

Example: Runlevel 0 Scripts

```
pete@icebox:/etc/rc.d/rc0.d$ ls
K10updates  K80openvpn
```

This shows that during shutdown (runlevel o), the system will first kill updates service (K10), then openvpn service (K80)

Configuration

- **Default runlevel:** Set in `/etc/inittab` file
 - **Modification:** Can change default runlevel in same file
-

2. System V Service Management

Common Service Commands

```
# List all services
$ service --status-all

# Start a service
$ sudo service networking start

# Stop a service
$ sudo service networking stop

# Restart a service
$ sudo service networking restart
```

Important Notes

- These commands work with both Sys V and Upstart services
 - Commands support transition from traditional Sys V init scripts
 - Cross-compatibility maintained for legacy support
-

3. Upstart Overview

Background and Purpose

- **Developer:** Canonical
- **Usage:** Previously used on Ubuntu (now replaced by systemd)
- **Improvement over Sys V:** Addresses strict startup processes and task blocking
- **Architecture:** Event and job-driven model

Identification (Upstart)

- **How to identify:** Check if `/usr/share/upstart` directory exists

Core Concepts

- **Jobs:** Actions that Upstart performs
- **Events:** Messages from other processes that trigger jobs

Job Configuration Location

```
pete@icebox:~$ ls /etc/init
acpid.conf               mountnfs.sh.conf
alsa-restore.conf       mtab.sh.conf
alsa-state.conf          networking.conf
alsa-store.conf          network-interface.conf
anacron.conf             network-interface-container.conf
```

Example Configuration: networking.conf

```
start on runlevel [235]
stop on runlevel [0]
```

This configuration starts networking on runlevels 2, 3, or 5, and stops on runlevel 0.

Upstart Boot Process

1. Load job configurations from `/etc/init`
 2. Wait for startup event
 3. Run jobs triggered by the event
 4. Jobs create new events
 5. New events trigger more jobs
 6. Continue until all necessary jobs complete
-

4. Upstart Job Management

View All Jobs

```
initctl list
```

Example output:

```
shutdown stop/waiting
console stop/waiting
...
```

Format: [job_name][goal]/[current_status]

View Specific Job Status

```
initctl status networking
```

Example output:

```
networking start/running
```

Manual Job Control

```
# Start a job
$ sudo initctl start networking

# Stop a job
$ sudo initctl stop networking

# Restart a job
$ sudo initctl restart networking

# Emit an event
$ sudo initctl emit some_event
```

Job Status Interpretation

- **First field:** Job name
- **Second field** (before /): Goal of the job

- **Third field** (after /): Current status
-

5. Systemd Overview

Background and Identification

- **Status**: Emerging standard for init systems
- **How to identify**: Check if `/usr/lib/systemd` directory exists

Core Architecture

- **Goals-based**: Uses targets to achieve system state
- **Flexible**: Non-sequential process startup
- **Robust**: Handles complex dependencies

Configuration Locations

- `/etc/systemd/system`
- `/usr/lib/systemd/system`

Boot Process

1. Load configuration files
2. Determine boot goal (usually `default.target`)
3. Figure out dependencies of boot target
4. Activate dependencies

Systemd Targets (equivalent to runlevels)

- **poweroff.target**: Shutdown system
- **rescue.target**: Single user mode
- **multi-user.target**: Multiuser with networking
- **graphical.target**: Multiuser with networking and GUI
- **reboot.target**: Restart
- **default.target**: Usually points to graphical.target

Unit Types

- **Service units**: Services (.service files)
- **Mount units**: Filesystems (.mount files)

- **Target units:** Groups other units (.target files)
-

6. Systemd Unit Management

Example Unit File: `foobar.service`

```
[Unit]
Description=My Foobar
Before=bar.target

[Service]
ExecStart=/usr/bin/foobar

[Install]
WantedBy=multi-user.target
```

Unit File Sections

- **[Unit]:** Description and ordering control
- **[Service]:** Start, stop, reload configuration
- **[Install]:** Dependency configuration

Systemd Commands

```
# List all units
$ systemctl list-units

# View unit status
$ systemctl status networking.service

# Start a service
$ sudo systemctl start networking.service

# Stop a service
$ sudo systemctl stop networking.service

# Restart a service
$ sudo systemctl restart networking.service

# Enable a unit (start at boot)
$ sudo systemctl enable networking.service
```

```
# Disable a unit (don't start at boot)
$ sudo systemctl disable networking.service
```

7. Power State Management

Shutdown Commands

```
# Shutdown immediately
$ sudo shutdown -h now

# Shutdown in 2 minutes
$ sudo shutdown -h +2

# Restart immediately
$ sudo shutdown -r now

# Alternative restart command
$ sudo reboot
```

Command Parameters

- **-h**: Halt (power off) the system
- **-r**: Restart the system
- **now**: Execute immediately
- **+[minutes]**: Execute after specified minutes

Use Cases

- **System maintenance**: Controlled shutdown for updates
 - **Remote administration**: Scheduled shutdowns
 - **Emergency situations**: Immediate system halt
-

Summary Comparison

Feature	System V	Upstart	Systemd
Boot Process	Sequential	Event-driven	Parallel/Goal-based
Configuration	<code>/etc/inittab</code>	<code>/etc/init/*.conf</code>	<code>/etc/systemd/system</code>
Performance	Slow	Better	Best
Dependency	Script-based	Event-based	Target-based
Current Usage	Legacy	Deprecated	Modern standard

Key Takeaways

1. **System V** is the traditional, sequential init system
2. **Upstart** improved upon Sys V with event-driven architecture
3. **Systemd** is the modern standard with parallel processing and robust dependency management
4. All systems support similar service management commands for compatibility
5. Understanding runlevels/targets is crucial for system administration